arXiv:2107.13872v1 [quant-ph] 29 Jul 2021

Quantum Arithmetic for Directly Embedded Arrays

Alberto Manzano[a*†], Daniele Musso[b*†], Álvaro Leitao[a], Andrés Gómez[b] ,

Carlos Vázquez[a], Gustavo Ordóñez[c] and María Rodríguez-Nogueiras[c].

[a] *Department of Mathematics and CITIC, Universidade da Coruña, A Coruña, Spain*

[b] *Galicia Supercomputing Center (CESGA), Santiago de Compostela, Spain*

[c] *Global Risk Analytics, HSBC, United Kingdom*

Abstract

We describe a general-purpose framework to design quantum algorithms relying upon an efficient handling of arrays. The corner-stone of the framework is the direct embedding of information into quantum amplitudes, thus avoiding the need to deal with square roots or encode the information in registers. We discuss the entire pipeline, from data loading to information extraction. Particular attention is devoted to the definition of an efficient tool-kit of quantum arithmetic operations on arrays. We comment on strong and weak points of the proposed manipulations, especially in relation to an effective exploitation of quantum parallelism. Eventually, we give explicit examples regarding the manipulation of generic oracles.

* *Corresponding authors:* alberto.manzano.herrero@udc.es *and* daniele.musso@cesga.es

† *These authors contribute equally to this work.*

Contents

1 Introduction, motivation and main results **3**

2 Data loading **5**

3 Quantum arithmetic **6**
3.1 Ordering . . . 6
3.1.1 Reversing . . . 6
3.1.2 Permutations . . . 7
3.1.3 Cyclic permutations . . . 9
3.2 Addition . . . 9
3.2.1 Sum . . . 9
3.2.2 Reductions . . . 10
3.3 Products . . . 10
3.3.1 Multiplication by a constant . . . 10
3.3.2 Array multiplication . . . 11
3.3.3 Squaring and scalar product . . . 12

4 Information extraction **12**
4.1 Amplified iterations . . . 13

5 Examples **14**
5.1 Constant shift of a given oracle . . . 14
5.2 Approximated linear shift of a given oracle . . . 15

6 Discussion and conclusion **16**

7 Acknowledgements **16**

A Details on data loading **17**
A.1 Pointwise loading of a matrix . . . 17
A.2 Loading a constant array . . . 18

1 Introduction, motivation and main results

Quantum hardware and software are still in their early days of development, thus the design of quantum algorithms typically focuses on low-level operations. Although one should always keep in mind the hardware limitations, especially when describing possible near-term implementations of quantum algorithms, it is convenient to pursue higher levels of abstraction. Apart from its long-term and algorithmic interest, a more abstract and standardized approach serves practical purposes too, for example that of making the benchmarking of quantum computer performances a more solid and transparent process. In turn, this helps pushing the research and the development in quantum computation at all levels.

In the present paper, we describe a novel framework for the design of quantum algorithms on a more abstract plane. To this aim, our first proposal consists in the definition of a *quantum matrix*, namely a quantum state organized in two registers:

$$|\psi\rangle = \sum_{i=0}^{I-1} \sum_{j=0}^{J-1} c_{ij} \, |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ , \tag{1}$$

where $|\rangle_{n_J}$ indicates a register composed of n_J qubits corresponding to $J = 2^{n_J}$ states, while $|\rangle_{n_I}$ is a register composed of n_I qubits corresponding to $I = 2^{n_I}$ states. The overall state $|\psi\rangle$, as defined in (1), is manifestly presented with the structure of a matrix; specifically, we interpret i as the index running over the rows and j as the index running over the columns. The rightmost qubit within a register is associated with the least significative digit of the associated index, in binary notation.[1] This way of storing the information has a common ground with that of Flexible Representation of Quantum Images (FRQI) and the Novel Enhanced Quantum Representation (NEQR) [Le+11] [Zha+13]. The main difference with FRQI and NEQR is that we codify the information of the (i, j) entry of the matrix in the quantum amplitude c_{ij}.Intuitively, the matrix (1) is a bi-dimensional memory array where c_{ij} encodes the information stored in the $|i\rangle_{n_I} \otimes |j\rangle_{n_J}$ memory location (see Figure 1).

	$\|0\rangle_{n_J}$	$\|1\rangle_{n_J}$	...	$\|j\rangle_{n_J}$	...	$\|J\rangle_{n_J}$
$\|0\rangle_{n_I}$	c_{00}	c_{01}	...	c_{0j}	...	c_{0J}
$\|1\rangle_{n_I}$	c_{10}	c_{11}	...	c_{1j}	...	c_{1J}
...	...	...	...	...	...	...
$\|i\rangle_{n_I}$	c_{i0}	c_{i1}	...	c_{ij}	...	c_{iJ}
...	...	...	...	...	...	...
$\|I\rangle_{n_I}$	c_{I0}	c_{I1}	...	c_{Ij}	...	c_{IJ}

Figure 1: Quantum matrix structure.

The second proposal that we describe in the present work is a key technical feature about how we encode the information into the quantum amplitudes, the so-called *direct embedding* [Kub+20]. Namely, the information to be stored into the quantum matrix is directly loaded into the amplitudes without taking square roots, as it is instead usually done in the literature. Such loading choice has several and important implications in later stages of the quantum algorithms and –most importantly– the information stored into the quantum state is handled and combined more easily, because algebraic operations are not hampered by the presence of square roots. This allows us to define an "arithmetic library" composed of many fundamental arithmetic operations to handle arrays stored into the quantum matrix in an efficient manner. Such "general purpose" library provides a versatile framework for the implementation of wide classes of algorithms. In this work, we provide some simple example algorithms without aiming to be exhaustive. The possibility of implementing arithmetic operations within a quantum framework has been considered in the literature since the early days of quantum computation. Apart from the

[1] Thus, we are adopting a little-endian convention.

quantum implementation of logical circuits corresponding to basic operations, like the quantum adder [Dra00; Cuc+04], also the manipulation of "continuous" numbers has been studied. Let us mention some works which, at least in spirit, are closer to ours [Wan+20; VBE96; LB99; XY19]. The difference with such approaches consists in the fact that we use a new embedding and organize the information into a matrix (58); these two aspects combined allow us to work in a transparent and simple manner. For the same reason, the extraction of the information at the end of the quantum circuit requires strategies adapted to our encoding.

The third and final proposal in this paper is to give a full overview of the complete pipeline, or overall structure, of the generic algorithm admitting implementation within this framework. The first step of every algorithm corresponds to loading some input data. In the quantum case, it is often convenient to split this step into two sub-steps:

- loading a probability distribution p_{ij} [GR02a][Nak+21][Kub+20]
- loading a bi-dimensional function f_{ij} (possibly by means of methods that load information a line at a time).[2]

It is not strictly necessary to split the loading into two steps. Yet, we consider such splitting because –typically– we adopt different loading techniques for them: the probability distribution is loaded with a state preparation algorithm (*e.g.* a multiplexor binary tree); the function is loaded by means of an auxiliary qubit meant to tell "good" and "bad" states apart. We describe the first step of the pipeline in Section 2.

In Section 3 we describe the second step of the pipeline corresponding to the implementation of various arithmetic operations, typically at the level of entire arrays or sub-arrays, and we refer to it as *quantum arithmetic*. In Section 4 we describe the last step of the pipeline, which corresponds to extracting the information that we have stored in the quantum state, namely the read-out of the state that encodes the result of the algorithm.

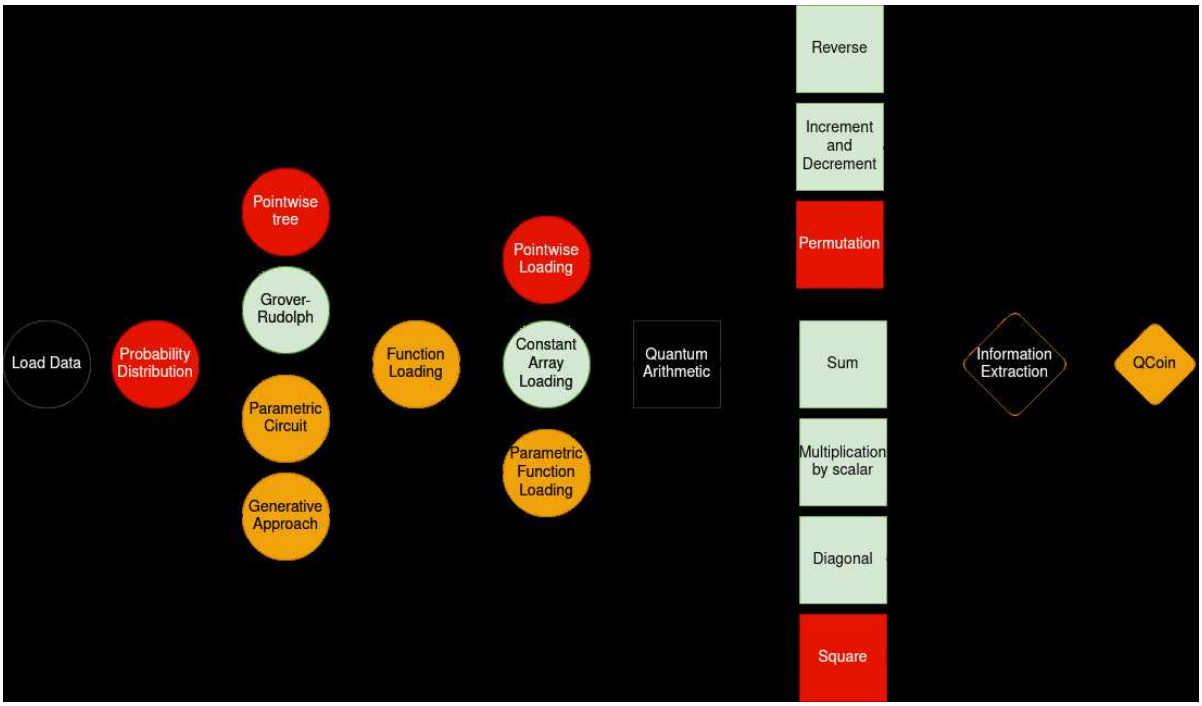

Figure 2: Diagramatic structure of the pipeline. The dark grey nodes represent the three main steps; the nodes in red corresponds to algorithms that are not efficient, for those in yellow the efficiency varies from case to case and finally the nodes in green represent efficient algorithms.

[2] In reference to the notation introduced for the quantum matrix (1), we have $c_{ij} = p_{ij} f_{ij}$, where the indexes are not summed over.

One of the advantages of organizing the pipeline as in Figure 2 is that it enjoys a modular structure, therefore we can develop and analyse each of the steps independently, achieving a better understanding of the problems in each domain. The color coding corresponds to the efficiency of the single modules.[3] In particular, an overall efficient algorithm would correspond to an end-to-end green path from left to right across the diagram. In searching for possible implementations for a desired algorithm, the challenge is to improve the necessary blocks so to follow a completely green path.

2 Data loading

Data loading is a generic step which is required essentially in any quantum algorithm. The actual data to be loaded can vary in nature and serves different purposes. The data can correspond –for example– to the discretization of a normalized general real function f defined on a two-dimensional domain. This is, for instance, the typical setup needed for many tasks in mathematical finance, where the two dimensions represent an underlying price and time, respectively. Having in mind applications to finance, we focus on the loading of a real function upon a probability distribution. Yet, we can as well think of the loading of more general data corresponding to a complex matrix, as long as the normalization of the quantum state is respected.

The recipe described in the appendix works in a pointwise fashion, exploiting an auxiliary register to store the desired value into the quantum amplitude at each "memory address", namely, to store it into the associated entry of the quantum matrix. It is important to underline from the outset that this pointwise approach is generically not efficient. In order to attain efficiency at the level of the full algorithm, we need to assume that the loading procedure can be implemented in an alternative and efficient way; in other words, we need to assume the existence of a suitable efficient oracle. Nonetheless, as we will show and stress later, a set of efficient manipulations for generic arrays is possible even when their loading is not efficient. This observation stems directly from the modular structure described in the pipeline of Figure 2.

There are two different aspects related to the complexity of the state preparation, the quantum circuit complexity, on one side, and the complexity of the pre-processing algorithm (where needed), on the other. The former expresses a count of quantum operations or some quantitative estimation of the depth of the quantum loading circuit, the latter refers to the possible pre-processing needed to compute the case-specific values for the parameters of the quantum loading circuit.[4] Here, we are going to discuss only the former, namely, the circuit complexity. To this purpose, we adopt the customary complexity indicator which simply relies on a count of the necessary CNOT gates. This is motivated by the fact that CNOTs are sensibly more error-prone and require a longer execution time than single-qubit gates, as commented -for instance- in [SBM06].

Loading a generic real array is not a trivial problem. In Appendix A.1 we referred to a pointwise loading, without worrying about its optimization.[5] To this regard, the state of the art is currently set by two alternative approaches [Bru20]: one based on multiplexors [MV05; SBM06] and the other one based on Schmidt's decomposition [PB11; Pat13]. Both approaches give essentially the same leading CNOT complexity, namely, a number of CNOTs which scales as 2^{n+1} for the preparation of the generic n-qubit state.

Let us stress that, in the very specific case where we need to load a constant array, the procedure of Appendix A.2 requires (in the worst case scenario) n_I x-gates, two y-rotations and one multi-controlled NOT gate. Such numbers must be compared with their classical counterpart, where the loading of a constant array on a line of the $I \times J$ matrix requires J operations, considering that the process of copying a single number from memory is an operation. Therefore, the loading of a constant function is more interesting from the quantum speed-up perspective than the pointwise loading of a generic function.

[3] It should be stressed that the efficiency of the single modules depicted in the diagram of Figure 2 refers to the current state of the art which can change in the future, that is, it does not necessarily represent a structural limitation.

[4] An example of pre-processing algorithm would be the computation of the values of the angles in a tree-like loading (see for instance [GR02b; Her21]).

[5] Despite it not being optimal, we have adopted the pointwise method for its simplicity. We postpone the study of more optimized approaches to the future.

Indeed, in principle, we need exponentially fewer operations on a quantum computer to load a constant array. Interestingly, the number of operations needed does *not* depend on the length of the constant array that we want to load, but it does depend on the number of rows of the matrix that we have to control. Here we can directly see the nature of quantum systems in practice, there is an "extra" cost associated to acting on a single element of the system without impacting the others. That makes operations on single elements inefficient and operations on the whole structure very efficient.

3 Quantum arithmetic

In the present section we provide a collection of tools for the efficient arithmetic handling of arrays encoded into a quantum matrix through direct embedding. These tools have been implemented and tested using Qiskit [ANI+21]. We present a selected set of implemented operations. Other operations potentially implementable within this framework are –for instance– those described in [CW12].

3.1 Ordering

The first operations that we introduce are those which allow us to move elements within the quantum matrix. Manipulating single elements in the matrix has a much higher cost than performing operations on the whole structure. For this reason, we first introduce a global reversing operation and then we introduce generic permutations.

3.1.1 Reversing

By reversing, we mean the operation

$$R : (c_{i,0}, c_{i,1}, c_{i,2}, c_{i,3}, ..., c_{i,J-1}) \longrightarrow (c_{i,J-1}, c_{i,J-2}, c_{i,J-3}, ..., c_{i,0}) \ , \tag{2}$$

where, for concreteness, we have addressed the reversing operation on the i-th row of the quantum matrix. Note that it is straightforward to perform the reversing operation on a column.

We divide the process in three steps:

- Mask the row. In this case we only need to mask the register corresponding to the row (the $|\rangle_{n_I}$ register) and leave the column register untouched. For more information on this operation see Appendix A.1.
- Apply n_J controlled x-gates. The controlled qubits are those of the row register. The target qubits are those of the column register.
- Undo step one, by applying again the same masking operation as before.

Following the steps above, we can perform a reversing operation on any row of the quantum matrix.

If we wanted instead to reverse the whole matrix, the operation would be more efficient than just reversing a row or a column.[6] In that case, there is no need to control on any qubit, we just need to apply an x-gate to each register of the quantum matrix.

As an explicit example, let us think of an $I \times J$ quantum matrix and, for simplicity, let us consider $I = 2$. Hence, we need $n_I = 1$ and $n_J = \log_2(J)$. Suppose that we have loaded the following quantum matrix:

$$|\chi_1\rangle = \frac{1}{A\sqrt{IJ}} \left(\sum_{j=0}^{J-1} c_{0j} |0\rangle_{n_I} \otimes |j\rangle_{n_J} + \sum_{j=0}^{J-1} c_{1j} |1\rangle_{n_I} \otimes |j\rangle_{n_J} \right) . \tag{3}$$

In order to reverse the first row, we start by applying an x-gate to the row register obtaining the state:

$$|\chi_2\rangle = \frac{1}{A\sqrt{IJ}} \left(\sum_{j=0}^{J-1} c_{0j} |1\rangle_{n_I} \otimes |j\rangle_{n_J} + \sum_{j=0}^{J-1} c_{1j} |0\rangle_{n_I} \otimes |j\rangle_{n_J} \right) . \tag{4}$$

[6] The inversion of the entire matrix corresponds to reading its entries from the lower-right corner *i.e.* in the opposite direction to how matrix entries are usually read.

Now, the row on which we are focusing, namely the one corresponding to c_{0j} for $j = 0, ..., J-1$, has all the qubits in the row register set to one (in this case the row register is just the qubit $|\rangle_{n_I}$). So, by means of controlled operations on $|\chi_2\rangle$, we act only on the row c_{0j}. Specifically, we apply an x-gate controlled on the row register, which acts on all the qubits of the column register. This yields

$$|\chi_3\rangle = \frac{1}{A\sqrt{IJ}} \left(\sum_{j=0}^{J-1} c_{0j}|1\rangle_{n_I} \otimes |J-1-j\rangle_{n_J} + \sum_{j=0}^{J-1} c_{1j}|0\rangle_{n_I} \otimes |j\rangle_{n_J} \right) . \tag{5}$$

Finally, we apply again an x-gate to the row register obtaining:

$$|\chi_4\rangle = \frac{1}{A\sqrt{IJ}} \left(\sum_{j=0}^{J-1} c_{0j}|0\rangle_{n_I} \otimes |J-1-j\rangle_{n_J} + \sum_{j=0}^{J-1} c_{1j}|1\rangle_{n_I} \otimes |j\rangle_{n_J} \right) . \tag{6}$$

The last step consists in undoing the mask.

3.1.2 Permutations

Permutations of two elements of an array, *i.e.* swaps of two entries, are demanding operations as we have to manipulate individual elements, instead of whole blocks in the quantum matrix. For the sake of simplicity, in what follows we discuss the algorithm referring to a quantum matrix given by a single row. Generalizing to larger matrices is straightforward. It is relevant to point out that also the extension to higher dimensionalities, from bi-dimensional matrices to d-dimensional tensors, is doable, yet it requires additional controlled operations.[7] Specifically, consider the state:

$$|\phi_1\rangle = \frac{1}{\|f\|_\infty \sqrt{J}} \sum_{j=0}^{J-1} f_j |j\rangle_{n_J} \ , \tag{7}$$

and let us write it in the notation:

$$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1}), \tag{8}$$

which is more convenient to understand how the different gates act on the order of the components. The strategy presented here to perform a permutation of two arbitrary elements in (8) consists in using a pivot. That is, we choose a fixed position k (pivot) and implement the permutations of the component placed at position k and any other component in the array. Once this is done, the generic swap of two elements can be obtained by means of three operations, at most. For example, if we aim to make a permutation of elements in positions i and j in

$$(f_0, ..., f_i, ..., f_k, ..., f_j, ..., f_{J-1}) \ , \tag{9}$$

we would need to perform the following three steps: First, we permute the positions $j \Longleftrightarrow k$, obtaining

$$(f_0, ..., f_i, ..., f_j, ..., f_k, ..., f_{J-1}) \ . \tag{10}$$

Then, we consider the permutation of positions $i \Longleftrightarrow k$ corresponding to the permutation of elements $i \Longleftrightarrow j$:

$$(f_0, ..., f_j, ..., f_i, ..., f_k, ..., f_{J-1}) \ . \tag{11}$$

Finally, we perform again step one, obtaining the desired permuted state, namely

$$(f_0, ..., f_j, ..., f_k, ..., f_i, ..., f_{J-1}) \ . \tag{12}$$

Now, the key in the algorithm is to understand how to actually perform in practice the permutations with the pivot. They can be implemented through x-gates and controlled x-gates. Moreover –without

[7] Note that the additional controlled operations may result in additional complexity.

losing generality– we choose the last element of the register as the pivot. If we have n_J qubits, the single x-gates acting on state (8) have the effects described in Table 1.

Gate	Old State	New State
$\mathbb{1}^{\otimes n_J-1} \otimes X$	$f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$f_1, f_0, f_3, f_2, ..., f_{J-3}, f_{J-4}, f_{J-1}, f_{J-2})$
$\mathbb{1}^{\otimes n_J-2} \otimes X \otimes \mathbb{1}$	$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$(f_2, f_3, f_0, f_1, ..., f_{J-2}, f_{J-1}, f_{J-4}, f_{J-3})$
...	...	...
$X \otimes \mathbb{1}^{\otimes n_J-1}$	$(f_0, f_1, f_2, f_3, ..., f_{J/2-1}, f_{J/2}, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$(f_{J/2}, f_{J/2+1}, f_{J/2+2}, f_{J/2+3}, ..., f_{J-1}, f_0, ..., f_{J/2-4}, f_{J/2-3}, f_{J/2-2}, f_{J/2-1})$

Table 1: Note how each gate affects the whole state, however the coloring emphasizes the effect of the gates on the rightmost blocks.

From Table 1 we can see that the single x-gates are performing swaps of blocks of *contiguous memory positions*. When we act on more significant qubits we are affecting bigger blocks and each gate is affecting the whole state. In this algorithm we are only interested on the effect that the gate has on certain blocks of the array (the ones highlighted). Using multi-controlled x-gates where the controls are applied to all qubits (except the one where we apply the x gate) and acting on state (8), we get the results reported in Table 2.

Gate	Old State	New State
$c^{\otimes n_J-1} \otimes X$	$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-1}, f_{J-2})$
$c^{\otimes n_J-2} \otimes X \otimes c$	$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$(f_0, f_1, f_2, f_3, ..., f_{J-4}, f_{J-1}, f_{J-2}, f_{J-3})$
...	...	...
$X \otimes c^{\otimes n_J-1}$	$(f_0, f_1, f_2, f_3, ..., f_{J/2-1}, f_{J/2}, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J-1})$	$(f_0, f_1, f_2, f_3, ..., f_{J-1}, f_{J/2}, ..., f_{J-4}, f_{J-3}, f_{J-2}, f_{J/2-1})$

Table 2: The use of controlled gates has two different effects. In the second column of the table, we are emphasizing that they connect certain blocks of the matrix (those highlighted). In the third column of the table, we are emphasizing the fact that only the rightmost entries of each block are impacted.

In this case it is clear that the effect of the controlled operations is to permute the last elements of each highlighted block.

We need to combine both operations, x-gates and multi-controlled x-gates, to perform the permutation of any element with the pivot (the last element, according to our choice). The strategy can be implemented recursively in the following way:

1. Move the last element of the array to the block where the element we wish to permute is located. This is done through a suitable multi-controlled x-gate.

2. If at this point the two elements that we wanted to interchange have been actually swapped, then undo all previous operations (both x-gates and multi-controlled x-gates) except for the last one and finish. These operations are needed to bring back to their original position all the other elements except the pair that has been swapped. Otherwise continue.

3. Swap the blocks on which we have acted at step 1. This is done through a single x-gate and serves the purpose of moving to the right the block on which we need to focus.

4. Go back to step 1.

For the sake of clarity, let us give a simple explicit example. Consider the state

$$|\psi\rangle = (f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7) \ , \tag{13}$$

and suppose we want to permute the first element f_0 with the pivot element f_7. We can proceed as follows:

$$
\begin{aligned}
(X \otimes c \otimes c) \left(f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7 \right) &= \left(f_0, f_1, f_2, f_7, f_4, f_5, f_6, f_3 \right), \\
(X \otimes \mathbb{1} \otimes \mathbb{1}) \left(f_0, f_1, f_2, f_7, f_4, f_5, f_6, f_3 \right) &= \left(f_4, f_5, f_6, f_3, f_0, f_1, f_2, f_7 \right), \\
(c \otimes X \otimes c) \left(f_4, f_5, f_6, f_3, f_0, f_1, f_2, f_7 \right) &= \left(f_4, f_5, f_6, f_3, f_0, f_7, f_2, f_1 \right), \\
(\mathbb{1} \otimes X \otimes \mathbb{1}) \left(f_4, f_5, f_6, f_3, f_0, f_7, f_2, f_1 \right) &= \left(f_6, f_3, f_4, f_5, f_2, f_1, f_0, f_7 \right), \\
(c \otimes c \otimes X) \left(f_6, f_3, f_4, f_5, f_2, f_1, f_0, f_7 \right) &= \left(f_6, f_3, f_4, f_5, f_2, f_1, f_7, f_0 \right).
\end{aligned}
$$

Now that we have effectively swapped the element 0 and the element 7 we just have to relocate the rest of the elements (Step 3).

$$
\begin{aligned}
(\mathbb{1} \otimes X \otimes \mathbb{1}) \left(f_6, f_3, f_4, f_5, f_2, f_1, f_7, f_0 \right) &=, \left(f_4, f_5, f_6, f_3, f_7, f_0, f_2, f_1 \right), \\
(c \otimes X \otimes c) \left(f_4, f_5, f_6, f_3, f_7, f_0, f_2, f_1 \right) &= \left(f_4, f_5, f_6, f_3, f_7, f_1, f_2, f_0 \right), \\
(X \otimes \mathbb{1} \otimes \mathbb{1}) \left(f_4, f_5, f_6, f_3, f_7, f_1, f_2, f_0 \right) &= \left(f_7, f_1, f_2, f_0, f_4, f_5, f_6, f_3 \right), \\
(X \otimes c \otimes c) \left(f_7, f_1, f_2, f_0, f_4, f_5, f_6, f_3 \right) &= \left(f_7, f_1, f_2, f_3, f_4, f_5, f_6, f_0 \right).
\end{aligned}
$$

3.1.3 Cyclic permutations

Cyclic permutations corresponds to the two transformations:

$$
\text{Left} : (f_0, f_1, f_2, ..., f_{J-2}, f_{J-1}) \longrightarrow (f_1, f_2, f_3, ..., f_{J-1}, f_0) \ , \tag{14}
$$

$$
\text{Right} : (f_0, f_1, f_2, ..., f_{J-2}, f_{J-1}) \longrightarrow (f_{J-1}, f_0, f_1, ..., f_{J-3}, f_{J-2}) \ , \tag{15}
$$

where we follow the same notation adopted in Section 3.1.2.[8] These operators have been discussed in depth in [Li+14] and their implementation can be immediately extended to our framework, upon adding suitable controls.

3.2 Addition

In this subsection we discuss both the sum of whole arrays and the sum of their components (reduction).

3.2.1 Sum

Consider the state $|\psi_1\rangle$ given in (61):

$$
|\psi_1\rangle \supset \frac{1}{\|f\|_\infty \sqrt{IJ}} \sum_{j=0}^{J-1} \sum_{i=0}^{I-1} f_{ij} |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ , \tag{16}
$$

where we have omitted the auxiliary register $|\rangle_a$ for convenience. Applying a Hadamard gate on the first qubit of the row register, namely

$$
|\psi_2\rangle = \left(\mathbb{1}^{\otimes n_I - 1} \otimes H \otimes \mathbb{1}^{\otimes n_J} \right) |\psi_1\rangle \ , \tag{17}
$$

[8] In what follows, we adopt the notation $\supset$ to consider just some specific terms that are relevant for our purposes within a bigger quantum state.

we get the sum and difference of the the rows grouped in pairs, explicitly

$$|\psi_2\rangle \supset \frac{1}{\sqrt{2}\|f\|_\infty \sqrt{IJ}} \sum_{j=0}^{J-1} \sum_{i=0}^{\frac{I}{2}-1} \Bigg[(f_{2i,j} + f_{2i+1,j}) \, |2i\rangle_{n_I} \otimes |j\rangle_{n_J} \\ + (f_{2i,j} - f_{2i+1,j}) \, |2i+1\rangle_{n_I} \otimes |j\rangle_{n_J} \Bigg] . \tag{18}$$

In the first row of (18), we get the sum of the first and second row of (16). In the second row of (18), instead, we get the difference between the first and the second row of (16). In the third row of (18), we get the sum of the third and fourth row of (16), and the same structure continues on.

An analogous sum/difference operation can be performed in columns. Note that, in order to consider the correct number of $\frac{1}{\sqrt{2}}$ factors, we need to count the Hadamard gates that we apply. Eventually, to sum two rows that are not in the same pair, we can take advantage of the permutations described in Section 3.1.2.

3.2.2 Reductions

By *reduction* we mean the summation of all the elements of an array

$$\text{Reduction} : (f_0, f_1, ..., f_{J-1}) \longrightarrow (f_0 + f_1 + ... + f_{J-1}, ...) \ , \tag{19}$$

where the result of the reduction is stored into the first entry of the array. Consider again the state defined in (16), that is,

$$|\psi_1\rangle \supset \frac{1}{\|f\|_\infty \sqrt{IJ}} \sum_{j=0}^{J-1} \sum_{i=0}^{I-1} f_{ij} |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ . \tag{20}$$

In order to perform a reduction by rows (*i.e.* summing the elements of each row and storing the result on the first column), we just need to apply a Hadamard gate to every qubit of the column register

$$|\psi_2\rangle = \left(\mathbb{1}^{\otimes n_I} \otimes H^{\otimes n_J}\right) |\psi_1\rangle \ , \tag{21}$$

which gives:

$$|\psi_2\rangle \supset \frac{1}{\sqrt{2}\|f\|_\infty \sqrt{IJ}} \sum_{i=0}^{I-1} \left(\sum_{j=0}^{J-1} f_{ij} \right) |i\rangle_{n_I} \otimes |0\rangle_{n_J} . \tag{22}$$

The parenthesis implies that we have the reduction of each row in the first column (which corresponds to $|0\rangle_{n_J}$). In the rest of the columns, we get other reductions with different combinations of signs, as implemented by the Walsh-Hadamard operator in (21).

If we were to do a reduction by columns, instead of by rows, we will need to apply the Walsh-Hadamard gate to the row register, instead of the column register. Correspondingly, we will get the reduction of the columns on the first row.

3.3 Products

In this subsection we consider the product of a whole array by a constant and the product of two arrays. Eventually, the scalar product of two arrays can be obtained composing the product of two arrays and a reduction. Similarly, the squaring of an array can be obtained as the product of the array by itself.

3.3.1 Multiplication by a constant

In order to multiply a row or a column by a constant, we need an extra qubit which we denote $|0\rangle_{\text{mul}}$. Consider the state defined in (61), but this time supplemented with the extra qubit:

$$|\psi_1\rangle \supset \frac{1}{\|f\|_\infty \sqrt{IJ}} \ |0\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} \sum_{i=0}^{I-1} f_{ij} |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ . \tag{23}$$

The multiplication operation consists merely in a controlled rotation. The rotation needs to be applied onto the auxiliary register $|0\rangle_{\text{mul}}$, it introduces a factor $\alpha = \cos\frac{\theta}{2}$, so we are initially restricted to multiplication by numbers between 0 and 1. This limitation can be circumvented by means of suitable manipulations of the normalization constants.

Depending on the controls that we apply, we can multiply a row, a column or a specific individual entry by α. For example, assume that we want to multiply the first row by α. In order to act solely on the first row, we first have to mask it by applying

$$|\psi_2\rangle = (X^{n_I} \otimes \mathbb{1}^{n_J})\,|\psi_1\rangle \ ; \tag{24}$$

explicitly, we obtain:

$$|\psi_2\rangle \supset \frac{1}{\|f\|_\infty \sqrt{IJ}}\ |0\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} f_{0j}|I-1\rangle_{n_I} \otimes |j\rangle_{n_J} + \dots \ . \tag{25}$$

The next step is to perform the controlled y-rotation

$$|\psi_3\rangle = \left[\mathcal{R}_y(\theta) \otimes C^{\otimes n_I} \otimes \mathbb{1}^{\otimes n_J}\right] |\psi_2\rangle \ , \tag{26}$$

where we have indicated the controls of the controlled rotations with the symbol C. Thus, (26) is to be interpreted as a controlled y-rotation acting on $|0\rangle_{\text{mul}}$ and controlled by the row register. Explicitly, (26) gives

$$\begin{aligned} |\psi_3\rangle \supset & \frac{1}{\|f\|_\infty \sqrt{IJ}} \cos\left(\frac{\theta}{2}\right) |0\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} f_{0j}|I-1\rangle_{n_I} \otimes |j\rangle_{n_J} \\ & + \frac{1}{\|f\|_\infty \sqrt{IJ}} \sin\left(\frac{\theta}{2}\right) |1\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} f_{0j}|I-1\rangle_{n_I} \otimes |j\rangle_{n_J} + \dots \ . \end{aligned} \tag{27}$$

Eventually, we have to unmask the state

$$|\psi_4\rangle = (X^{n_I} \otimes \mathbb{1}^{n_J})\,|\psi_3\rangle \ , \tag{28}$$

which yields

$$\begin{aligned} |\psi_4\rangle \supset & \frac{1}{\|f\|_\infty \sqrt{IJ}} \cos\left(\frac{\theta}{2}\right) |0\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} f_{0j}|0\rangle_{n_I} \otimes |j\rangle_{n_J} \\ & + \frac{1}{\|f\|_\infty \sqrt{IJ}} \sin\left(\frac{\theta}{2}\right) |1\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1} f_{0j}|0\rangle_{n_I} \otimes |j\rangle_{n_J} \\ & + \frac{1}{\|f\|_\infty \sqrt{IJ}} |0\rangle_{\text{mul}} \otimes \sum_{j=0}^{J-1}\sum_{i=1}^{I-1} f_{ij}|i\rangle_{n_I} \otimes |j\rangle_{n_J} \ . \end{aligned} \tag{29}$$

The relevant information is marked by $|0\rangle_{\text{mul}}$.

3.3.2 Array multiplication

In the present section, we describe the theoretical proposal for a more advanced operation: the multiplication of arrays. Its (overall) efficiency is related to that of the loading process. Let us assume to dispose of the following oracles:

$$O_f\big(|0\rangle \otimes |j\rangle\big) = f_j|0\rangle \otimes |j\rangle + \sqrt{1-f_j^2}\,|1\rangle \otimes |j\rangle, \tag{30}$$

$$O_g\big(|0\rangle \otimes |j\rangle\big) = g_j|0\rangle \otimes |j\rangle + \sqrt{1-g_j^2}\,|1\rangle \otimes |j\rangle, \tag{31}$$

which load the arrays f and g respectively. Morever, consider the swap operator:

$$S\big(|\alpha\rangle \otimes |\beta\rangle \otimes |j\rangle\big) = |\beta\rangle \otimes |\alpha\rangle \otimes |j\rangle. \tag{32}$$

To build the multiplication operator we are going to start from the state:

$$|\lambda_0\rangle = |0\rangle_g \otimes |0\rangle_f \otimes |j\rangle, \tag{33}$$

where we have two auxiliary qubits: $|0\rangle_f$ to load f and $|0\rangle_g$ to load g. First we load f:

$$|\lambda_1\rangle = \big(\mathbb{1} \otimes O_f\big)|\lambda_0\rangle = f_j|0\rangle_g \otimes |0\rangle_f \otimes |j\rangle + \sqrt{1-f_j^2}\,|0\rangle_g \otimes |1\rangle_f \otimes |j\rangle \tag{34}$$

In the second step we swap qubits $|\rangle_f$ and $|\rangle_g$:

$$|\lambda_2\rangle = S|\lambda_1\rangle = f_j|0\rangle_f \otimes |0\rangle_g \otimes |j\rangle + \sqrt{1-f_j^2}\,|1\rangle_f \otimes |0\rangle_g \otimes |j\rangle. \tag{35}$$

The third and last step consists in applying the oracle O_g:

$$\begin{aligned}|\lambda_3\rangle = \big(\mathbb{1} \otimes O_g\big)|\lambda_2\rangle &= g_j f_j|0\rangle_f \otimes |0\rangle_g \otimes |j\rangle + g_j\sqrt{1-f_j^2}\,|1\rangle_f \otimes |0\rangle_g \otimes |j\rangle \\ &+ \sqrt{1-g_j^2}\,f_j|0\rangle_f \otimes |1\rangle_g \otimes |j\rangle + \sqrt{1-g_j^2}\,\sqrt{1-f_j^2}\,|1\rangle_f \otimes |1\rangle_g \otimes |j\rangle.\end{aligned} \tag{36}$$

The multiplication of arrays f and g is encoded in the registers marked by $|0\rangle_f \otimes |0\rangle_g$:[9]

$$|\lambda_3\rangle = \big(\mathbb{1} \otimes O_g\big)\, S\,\big(\mathbb{1} \otimes O_f\big) \supset g_j f_j|0\rangle_f \otimes |0\rangle_g \otimes |j\rangle\ . \tag{37}$$

This procedure, when extended to the multiplication of more than two arrays. It is worth noticing the fact that this method depends on the loading complexity, that is, the efficiency of the employed oracles.

As a final comment, when in Section 1 we split the loading of the integrand $f \cdot p$ in two parts, that associated to the distribution and that associated to the function, we were in fact performing the multiplication of the two arrays $\{p_j\}$ and $\{f_j\}$.

3.3.3 Squaring and scalar product

The square of an array and the scalar product of two arrays can be obtained from operations that we have already defined above. The former is trivially just the multiplication of an array by itself. On the other hand, if we perform the reduction of the product of two arrays, we get their scalar product. As their construction depends on the steps commented in Section 3.3.2, the efficiency of the square of an array and the scalar product of two arrays is strongly dependent on the loading strategy for the arrays.

4 Information extraction

In this section we describe a method to extract the information stored in the quantum circuit at the end of the algorithm. The method is called Quantum Coin [AW99; SH20] (QCoin) and it can be applied in general to extract the absolute value of the amplitude a along a state $|\chi\rangle$. Although in this article we rely on the QCoin algorithm, there are other techniques which can be considered, such as those appearing in [Bra+00][Suz+20][Gri+20] [Giu+20].[10]

Let us suppose that we want to extract the amplitude of the state

$$|\chi\rangle = |1\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J}\ , \tag{38}$$

for some specific values i and j. For example, we want to read one entry of the quantum matrix defined in (1). The QCoin algorithm starts with an unamplified estimation of the amplitude along $|\chi\rangle$, which is

[9] In order to return back to the original ordering of the auxiliary qubits, $|0\rangle_g \otimes |0\rangle_f$, one can consider an extra swap S.

[10] Another information extraction method compatible with the direct embedding has been described in [Kub+20]; such method is tailored to compute expectation values of functions admitting a polynomial approximation.

then iteratively refined. Such unamplified estimation consists in a repeated set of identical experiments and measurements, which allows us to get an empirical value $\tilde{\mu}$ for the absolute value of the desired amplitude. This estimation provides a confidence interval

$$[\tilde{\mu} - \delta, \tilde{\mu} + \delta] \ , \tag{39}$$

relying on standard statistical tools, *e.g.* the Chebyshev's inequality. Then, one can pursue an iterative Grover amplification strategy, implemented as a "zoom-in" operation into the confidence interval estimated previously. This amounts to an iterative exponential refinement of the accuracy of the estimation of $\tilde{\mu}$ [AW99]. We give some additional details below.

4.1 Amplified iterations

An amplified stage consists of two operations:

- A constant shift of the amplitude a by the position of the lower bound of the confidence interval (39) of its empirically estimated value, namely

$$a \to \bar{a} = a - (\tilde{\mu} - \delta) \ . \tag{40}$$

  By construction, the shifted amplitude $\bar{a}$ is expected to have a positive value of the order of half the size of the confidence interval.[11]

- An enhancement of the probability P of measuring $|\chi\rangle$. Specifically, by means of Grover amplification, we obtain a state where the probability $\hat{P}$ of getting $|\chi\rangle$ is

$$\hat{P} = \gamma P \ , \tag{41}$$

  with $\gamma > 1$. An estimation of $\hat{P}$ with precision ϵ corresponds to an estimation of P with increased precision $\frac{\epsilon}{\gamma}$.

We define the Grover amplifier as usual

$$Q = O_{\bar{a}} \, R_\sigma \, O_{\bar{a}}^{-1} \, R_\chi \ , \tag{42}$$

where σ is the state

$$|\sigma\rangle \equiv |1\rangle_a \otimes |0\rangle_{n_I} \otimes |0\rangle_{n_J} \ , \tag{43}$$

and where $O_{\bar{a}}$ represents the operator which implements the algorithm followed by the shift (40). Note that the state $|\sigma\rangle$ is the entry marked as 00 in the quantum matrix (58). We denote with R_σ and R_χ the reflection operators given respectively by

$$R_\sigma = \mathbb{1} - 2|\sigma\rangle\langle\sigma| \ , \quad R_\chi = \mathbb{1} - 2|\chi\rangle\langle\chi| \ . \tag{44}$$

If we define the angle θ, such that

$$\sin(\theta) \equiv \langle\chi|\Psi\rangle \ , \tag{45}$$

where $|\Psi\rangle$ is the final quantum state, we have $P = \sin(\theta)^2$ and

$$\langle\chi|Q^k|\Psi\rangle = \sin[(2k+1)\theta] \ . \tag{46}$$

The corresponding amplification factor γ for the probability of measuring $|\chi\rangle$ is

$$\gamma = \frac{\sin[(2k+1)\theta]^2}{\sin(\theta)^2} \ . \tag{47}$$

In order to maximize γ we need to choose k so that

$$(2k+1)\theta \lesssim \frac{\pi}{2} \ . \tag{48}$$

[11] It is crucial to consider circumstances where the expected value is positive because the algorithm is insensitive to the sign. Similarly, note that –despite the loading module and the arithmetic manipulation module can be straightforwardly extended to complex amplitudes– the read-out would need to be phase sensitive.

5 Examples

In this section we give two explicit examples of manipulations for a given oracle.

5.1 Constant shift of a given oracle

Given an oracle O_f which loads the function f, we are interested in providing an efficient implementation of an oracle $O_{\bar{f}}$ for the shifted function

$$\bar{f}_j = f_j - s \ , \tag{49}$$

where s is a generic real constant. Note that constructing the oracle for $\bar{f}$ is non-trivial whenever f is non-constant.[12]

More specifically, consider the state

$$\begin{aligned} |\phi_1\rangle =& \frac{1}{\|f\|_\infty \sqrt{2J}} \sum_{j=0}^{J-1} \left(f_j |0\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} + \sqrt{1-f_j^2} |1\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_I} \right) \\ &+ \frac{1}{\sqrt{2J}} \sum_{j=0}^{J-1} \left(|0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} + |1\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \right) \ , \end{aligned} \tag{50}$$

which is the result of applying the oracle O_f to the initial state $|\psi_0\rangle$ given in (59). Note that the state $|\phi_1\rangle$ can be interpreted as having loaded the function f on the first row of the $2 \times J$ quantum matrix.

Following the steps described in Appendix A.2, we can load the constant function s into the second row of $|\phi_1\rangle$. Specifically, we load a constant array on the states associated to $i = 1$. It is important to remember that the quantum register $|i\rangle_{n_I}$ stores the row address of the arrays, while $|j\rangle_{n_J}$ stores the column address.

More explicitly, we obtain

$$\begin{aligned} |\phi_2\rangle =& \frac{1}{\|f\|_\infty \sqrt{2J}} \sum_{j=0}^{J-1} \left(f_j |0\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} + \sqrt{1-f_j^2} |1\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_I} \right) \\ &+ \frac{1}{\sqrt{2J}} \sum_{j=0}^{J-1} \left(s |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} + \sqrt{1-s^2} |1\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \right) \ . \end{aligned} \tag{51}$$

Following Section 3.2.1, we apply a Hadamard gate to the row qubit combining the two rows of the stored matrix. Namely

$$\begin{aligned} |\phi_3\rangle &= \left(\mathbb{1} \otimes H \otimes \mathbb{1}^{\otimes n_J} \right) |\phi_2\rangle \\ &= \frac{1}{2\|f\|_\infty \sqrt{J}} \sum_{j=0}^{J-1} \Bigg(f_j |0\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} + f_j |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \\ &+ \sqrt{1-f_j^2} |1\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} + \sqrt{1-f_j^2} |1\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \\ &+ s |0\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} - s |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \\ &+ \sqrt{1-s^2} |1\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} - \sqrt{1-s^2} |1\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \Bigg) \ . \end{aligned} \tag{52}$$

Let us focus just on the components $|0\rangle_a \otimes |1\rangle \otimes |j\rangle_{n_J}$, namely

$$|\phi_3\rangle \supset |\phi_3^{(-)}\rangle \equiv \frac{1}{2\|f\|_\infty \sqrt{J}} \sum_{j=0}^{J-1} (f_j - s) \, |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \ . \tag{53}$$

[12] The constant shift of a constant function amounts to just a global rotation. However, the constant shift of a non-constant function requires in general a position-dependent rotation.

This means that we have stored the difference array $f - s$, *i.e.* the array f shifted by the constant s, in the second line of the matrix. Note that, at the same time, we have stored the sum array in the first line,

$$|\phi_3\rangle \supset |\phi_3^{(+)}\rangle \equiv \frac{1}{2\|f\|_\infty\sqrt{J}} \sum_{j=0}^{J-1} (f_j + s)\, |0\rangle_a \otimes |0\rangle_1 \otimes |j\rangle_{n_J} \ . \tag{54}$$

5.2 Approximated linear shift of a given oracle

Let us consider a state $|\phi_4\rangle$ obtained by applying a CNOT operator to the state $|\phi_3\rangle$ where the contol qubit is the most significant digit in the register $|j\rangle_{n_J}$ and the target qubit is the row register. This operation flips the row index of the columns corresponding to the second half of the range $\{0, .., J-1\}$, namely those in the range $\{\frac{J}{2}, .., J-1\}$. We then have that the $|0\rangle_a \otimes |1\rangle \otimes |j\rangle_{n_J}$ components of $|\phi_4\rangle$ contain the function f shifted by $-s$ in the lower half of points and by s in the higher half, namely

$$\begin{aligned} |\phi_4\rangle \supset |\phi_4^{(-)}\rangle \equiv & \frac{1}{2\|f\|_\infty\sqrt{J}} \sum_{j=0}^{\frac{J}{2}-1} (f_j - s)\, |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \\ & + \frac{1}{2\|f\|_\infty\sqrt{J}} \sum_{j=\frac{J}{2}}^{J-1} (f_j + s)\, |0\rangle_a \otimes |1\rangle_1 \otimes |j\rangle_{n_J} \ . \end{aligned} \tag{55}$$

This corresponds to a shift by a step function, representing the most rudimentary approximation to a linear function.

So far, we have considered a two row matrix. However, if we want to iterate the step shift just described (for instance to the purpose of getting a finer approximation to a linear shift, we need to consider a larger matrix. Let us start over again with a matrix with $n_I = 2$ (that is, 4 rows) and repeat all the steps that led us to $|\phi_4\rangle$; we add a tilde to indicate that now we are working with a different matrix. We get

$$\begin{aligned} |\tilde{\phi}_4\rangle \supset |\tilde{\phi}_4^{(-)}\rangle \equiv & \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=0}^{\frac{J}{2}-1} (f_j - s)\, |0\rangle_a \otimes |01\rangle_2 \otimes |j\rangle_{n_J} \\ & + \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=\frac{J}{2}}^{J-1} (f_j + s)\, |0\rangle_a \otimes |01\rangle_2 \otimes |j\rangle_{n_J} \ . \end{aligned} \tag{56}$$

Now, let us load a constant array with the value $s' = \frac{2}{3}s$. We can then shift $|\tilde{\phi}_4\rangle$ by s' and then apply a CNOT operator whose control qubit is the second most significant digit in the column register $|j\rangle_{n_J}$ and whose target qubit is the most significant digit in the row register $|i\rangle_2$. This leads us to eventually choosing the components $|0\rangle_a \otimes |11\rangle_2 \otimes |j\rangle_{n_J}$, so we get

$$\begin{aligned} |\tilde{\phi}_5\rangle \supset |\tilde{\phi}_5^{(-)}\rangle \equiv & \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=0}^{\frac{J}{4}-1} \left(f_j - \frac{5}{3}s\right) |0\rangle_a \otimes |11\rangle_2 \otimes |j\rangle_{n_J} \\ & + \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=\frac{J}{4}}^{\frac{J}{2}-1} \left(f_j - \frac{1}{3}s\right) |0\rangle_a \otimes |11\rangle_2 \otimes |j\rangle_{n_J} \\ & + \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=\frac{J}{2}}^{\frac{3}{4}J-1} \left(f_j + \frac{1}{3}s\right) |0\rangle_a \otimes |11\rangle_2 \otimes |j\rangle_{n_J} \\ & + \frac{1}{2\|f\|_\infty\sqrt{2J}} \sum_{j=\frac{3}{4}J}^{J-1} \left(f_j + \frac{5}{3}s\right) |0\rangle_a \otimes |11\rangle_2 \otimes |j\rangle_{n_J} \ . \end{aligned} \tag{57}$$

The process can be iterated, however it becomes inefficient if the step-wise approximation of the linear shift is required to have a precision which, setting the number of discretisation points to N, scales as $\frac{1}{N}$.

6 Discussion and conclusion

The main goal of this work is to propose and describe a generic framework for the design of quantum algorithms based on direct embedding. Its modular structure, as depicted in Figure 2, is appealing and handy in a number of ways. For example, under this framework the main components of a quantum algorithm namely; data loading, arithmetic manipulations, and read-out can be studied and discussed separately. This holds true also for the considerations related to efficiency, whose current status is reflected into the color coding of Figure 2; specifically, an end-to-end efficient pipeline would be represented by a left-to-right path within the diagram that encounters only green boxes. Thus, the modular structure of the pipeline for the generic quantum algorithm helps to organise the research effort, compare and interpret different algorithms, and identify possible bottle-necks. Furthermore, it is possible to combine this framework with other existing routines. For instance, it is possible to adopt one's favourite amplitude amplification and estimation technique for the information-extraction part.

On a more technical level, the direct embedding of information into the quantum amplitudes avoids having to deal with square roots and thereby it opens the way to easier arithmetic manipulations of the data stored in the quantum state. In particular, we defined the *quantum matrix*, a two-dimensional array which can be thought of in analogy to a memory register: the basis states correspond to the row and column addresses of the memory locations, while the entries of the matrix are the quantum amplitudes representing the loaded information. As it has been previously illustrated, this construction allows for neat and flexible manipulation of arrays. We have also covered some basic arithmetic manipulations, for which we provided descriptions and implementation details. All in all, we set up a theoretical proposal for a package of arithmetic operations in a quantum framework. Its full potential and development requires further investigation and work, with particular focus on the loading and read-out modules.

Quantum matrices can be naturally generalized to multi-dimensional arrays. All the arithmetic manipulations proposed, as well as the loading and read-out techniques, can be extended in a straightforward way to the higher-dimensional and more general tensor setting. However, this comes at the cost of the potential necessity of additional controlled operations needed for "masking" the array and act only on a desired subset of entries. In other words, the cost of an operation is related to the co-dimension of the subset of entries to which it applies.

Finally, we also provided two specific example applications that are interesting on their own, beyond the discussions of the present work. Namely, the shift of a generic oracle by a constant and the shift by a step-wise approximate linear function. We note that their efficient implementation depends on the efficiency of the oracle to which the shift is applied. A constant shift for an oracle implements a vertical offset and it is useful –for example– in iterative algorithms where at each iteration an output oracle needs to be centered vertically, *i.e.* along the y axis.

7 Acknowledgements

All authors acknowledge the European Project NExt ApplicationS of Quantum Computing (NEASQC), funded by Horizon 2020 Program inside the call H2020-FETFLAG-2020-01(Grant Agreement 951821).

A. Manzano, Á. Leitao and C. Vázquez wish to acknowledge the support received from the Centro de Investigación de Galicia "CITIC", funded by Xunta de Galicia and the European Union (European Regional Development Fund- Galicia 2014-2020 Program), by grant ED431G 2019/01.

Part of the computational resources for this project were provided bt the Galician Supercomputing Center (CESGA).

We would like to thank Dario Ferraro, Simon Martiel and Javier Mas for fruitful discussions on some aspects of the present work.

A Details on data loading

A.1 Pointwise loading of a matrix

In this subsection we show how to load a generic matrix (*i.e.* a two dimensional array) into a quantum matrix (1) in a pointwise fashion. We will be considering states of the form:

$$|\hat{\psi}\rangle = \sum_{i=0}^{I-1}\sum_{j=0}^{J-1} c_{ij}|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ , \tag{58}$$

which corresponds to the quantum matrix introduced in (1) with the addition of an auxiliary register $|\rangle_a$. In what follows, let us assume for simplicity that the auxiliary register is one-dimensional, *i.e.* it consists of just one qubit. The other registers operate as described earlier when discussing Equation (1).

The pipeline of a quantum algorithm starts by loading an initial state. For example, this can represent a probability distribution and the most simple such case is the uniform distribution. Let us consider it explicitly. To load the uniform distribution, we apply the Walsh-Hadamard gate $\mathbb{1} \otimes H^{\otimes n_I} \otimes H^{\otimes n_J}$ to the base state $|0\rangle_a \otimes |0\rangle_{n_I} \otimes |0\rangle_{n_J}$, thus obtaining:

$$|\psi_0\rangle = \frac{1}{\sqrt{IJ}}\sum_{i=0}^{I-1}\sum_{j=0}^{J-1}|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J}. \tag{59}$$

Note that the loading of the distribution has not made use of the auxiliary qubit.

The next step in the pipeline is to load a real matrix f into the quantum matrix. To load a point f_{ij} in the corresponding register we need to act in such a way that we only impact the targeted quantum state. For that purpose we need have to perform three steps:

- Mask the state. Masking the state consists in converting the state $|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J}$ into the state $|0\rangle_a \otimes |I-1\rangle_{n_I} \otimes |J-1\rangle_{n_J} \equiv |0\rangle_a \otimes |11...11\rangle_{n_I} \otimes |11...11\rangle_{n_J}$. In terms of qubits, this requires to apply a NOT gate to all the qubits that are zero for the original state. The reason for this masking operation can be understood in the next step.
- Apply a suitable controlled y-rotation on the auxiliary qubit. The controls have to be applied on all the qubits except for the auxiliary one. Here we can see that the complexity of the algorithm depends on the number of qubits that we have to control. The angle for the rotation needs to be $\theta = \arccos\left(\frac{f_{ij}}{\|f\|_\infty}\right)$, where $\|f\|_\infty = \max(|f_{ij}|)$ is the infinity norm of the matrix f. The factor $\|f\|_\infty$ is needed to keep the amplitudes bounded so that the associated probabilities do not exceed 1.
- Undo step one. This consists in the application of the same mask already used at step one.

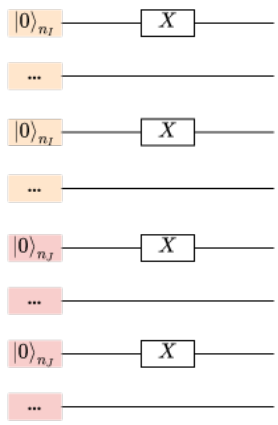

Figure 3: In yellow the row register. In red the column register.

Following this strategy we can load each of the values f_{ij}, thus getting the state:

$$|\psi_1\rangle = \frac{1}{\|f\|_\infty\sqrt{IJ}}\sum_{j=0}^{J-1}\sum_{i=0}^{I-1}\left(f_{ij}|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J} + \sqrt{1-f_{ij}^2}|1\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J}\right) . \tag{60}$$

Usually we focus only on the states marked with $|0\rangle_a$, namely

$$|\psi_1\rangle \supset \frac{1}{\|f\|_\infty\sqrt{IJ}}\sum_{j=0}^{J-1}\sum_{i=0}^{I-1} f_{ij}|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J} \ . \tag{61}$$

A.2 Loading a constant array

Loading a constant array follows pretty much the same strategy as the pointwise loading. For the purpose of giving an explicit example, we are going to describe the loading of a constant array, taking a real value $c \leq 1$, into a row of the quantum matrix. We start again loading a uniform distribution, thus obtaining (59). Then, we use a similar structure for loading the array as the one discussed before, namely

- Mask the state. In this case we only need to mask the register corresponding to the row (the $|\rangle_{n_I}$ register) and leave the column register untouched. As we mask only one register, we need fewer gates than for the pointwise loading described above.
- Apply a suitable controlled y-rotation on the auxiliary qubit. The angle for the rotation needs to be $\theta = \arccos(c)$. The controls have to be made only in the row registers. Here we can see that the number of controls to load a constant array is drastically reduced with respect to the generic function.
- Undo step one, by applying the same mask already considered there.

If we were to load the constant array in the register $|i\rangle_{n_I}$ we would get:

$$|\xi_1\rangle = ... + \frac{1}{A\sqrt{IJ}} \sum_{j=0}^{J-1} \left(c|0\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J} + \sqrt{1-c^2}|1\rangle_a \otimes |i\rangle_{n_I} \otimes |j\rangle_{n_J} \right) + ... \,, \tag{62}$$

where A is a normalization constant.

As it can be intuitively anticipated, the loading complexity grows together with the lack of symmetry of the loaded state. The constant case, being highly symmetric, is easy. In between the constant and the generic state with no symmetry, one can encounter lower degrees of symmetry, like for example functions which are piece-wise constant. We remind the reader that we adopted piece-wise constant functions in Subsection 5.2 to approximate a linear function and observed how the complexity grew with the approximation accuracy. For further discussions on the relation between the loading complexity and the symmetry of the loaded state we refer to [Bru20].

References

[VBE96] Vlatko Vedral, Adriano Barenco, and Artur Ekert. "Quantum networks for elementary arithmetic operations". In: *Phys. Rev. A* 54 (1 July 1996), pp. 147–153. DOI: `10.1103/PhysRevA.54.147`. URL: `https://link.aps.org/doi/10.1103/PhysRevA.54.147`.

[AW99] Daniel S. Abrams and C. P. Williams. "Fast Quantum Algorithms for Numerical Integrals and Stochastic Processes". In: *arXiv: Quantum Physics* (1999).

[LB99] Seth Lloyd and Samuel L. Braunstein. "Quantum Computation over Continuous Variables". In: *Phys. Rev. Lett.* 82 (8 Feb. 1999), pp. 1784–1787. DOI: `10.1103/PhysRevLett.82.1784`. URL: `https://link.aps.org/doi/10.1103/PhysRevLett.82.1784`.

[Bra+00] Gilles Brassard et al. "Quantum Amplitude Amplification and Estimation". In: *AMS Contemporary Mathematics Series* 305 (June 2000).

[Dra00] Thomas G. Draper. "Addition on a Quantum Computer". In: *arXiv e-prints*, quant-ph/0008033 (Aug. 2000), quant–ph/0008033. arXiv: `quant-ph/0008033 [quant-ph]`.

[GR02a] Lov Grover and Terry Rudolph. "Creating superpositions that correspond to efficiently integrable probability distributions". In: (Sept. 2002).

[GR02b] Lov Grover and Terry Rudolph. "Creating superpositions that correspond to efficiently integrable probability distributions". In: *arXiv e-prints*, quant-ph/0208112 (Aug. 2002), quant–ph/0208112. arXiv: `quant-ph/0208112 [quant-ph]`.

[Cuc+04] Steven A. Cuccaro et al. "A new quantum ripple-carry addition circuit". In: (2004). arXiv:quant-ph/0410184. arXiv: `0410184 [quant-ph]`.

[MV05] M. Mottonen and J. J. Vartiainen. "Decompositions of general quantum gates". In: *arXiv e-prints*, quant-ph/0504100 (Apr. 2005), quant–ph/0504100. arXiv: quant-ph/0504100 [quant-ph].

[SBM06] V.V. Shende, S.S. Bullock, and I.L. Markov. "Synthesis of quantum-logic circuits". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25.6 (Jan. 2006), pp. 1000–1010. ISSN: 1937-4151. URL: http://dx.doi.org/10.1109/TCAD.2005.855930.

[Le+11] Phuc Le et al. "A flexible representation of quantum images for polynomial preparation, image compression and processing operations, Quantum Inf". In: *Quantum Information Processing* 10 (Feb. 2011), pp. 63–84. DOI: 10.1007/s11128-010-0177-y.

[PB11] Martin Plesch and Caslav Brukner. "Quantum-state preparation with universal gate decompositions". In: *Physical Review A* 83.3 (Mar. 2011). ISSN: 1094-1622. URL: http://dx.doi.org/10.1103/PhysRevA.83.032302.

[CW12] Andrew M. Childs and Nathan Wiebe. "Hamiltonian Simulation Using Linear Combinations of Unitary Operations". In: *Quantum Info. Comput.* 12.11–12 (Nov. 2012), pp. 901–924. ISSN: 1533-7146.

[Pat13] Anirban Pathak. *Elements of Quantum Computation and Quantum Communication*. 1st. USA: Taylor & amp; Francis, Inc., 2013. ISBN: 1466517913.

[Zha+13] Yi Zhang et al. "NEQR: A novel enhanced quantum representation of digital images". In: *Quantum Information Processing* 12 (Aug. 2013). DOI: 10.1007/s11128-013-0567-z.

[Li+14] Xiaoyu Li et al. "A class of efficient quantum incrementer gates for quantum circuit synthesis". In: *International Journal of Modern Physics B* 28.01 (2014), p. 1350191. URL: https://doi.org/10.1142/S0217979213501919.

[XY19] Cui Xiaopeng and Shi Yu. "QBLAS: A Quantum Basic Linear Algebra and SimulationLibrary". In: *GitHub* (2019). URL: https://github.com/xpclove/qblas.

[Bru20] Timothée Brugière. "Methods for optimizing the synthesis of quantum circuits". PhD thesis. Université Paris-Saclay, Oct. 2020.

[Giu+20] Tudor Giurgica-Tiron et al. *Low depth algorithms for quantum amplitude estimation*. arXiv:2012.03348. 2020. arXiv: 2012.03348 [quant-ph].

[Gri+20] Dmitry Grinko et al. "Iterative Quantum Amplitude Estimation". In: *Quantum Information* 7.52 (2020).

[Kub+20] Kenji Kubo et al. "Variational quantum simulations of stochastic differential equations". In: (2020). arXiv: 2012.04429 [quant-ph].

[SH20] N. H. Shimada and Toshiya Hachisuka. "Quantum Coin Method for Numerical Integration". In: (2020). arXiv: 1910.00263 [quant-ph].

[Suz+20] Yohichi Suzuki et al. "Amplitude estimation without phase estimation". In: *Quantum Information Processing* 19.2 (Jan. 2020). ISSN: 1573-1332.

[Wan+20] Shengbin Wang et al. "Quantum Amplitude Arithmetic". In: (2020). arXiv: 2012.11056 [quant-ph].

[ANI+21] MD SAJID ANIS et al. *Qiskit: An Open-source Framework for Quantum Computing*. 2021. DOI: 10.5281/zenodo.2573505.

[Her21] Steven Herbert. "No quantum speedup with Grover-Rudolph state preparation for quantum Monte Carlo integration". In: *Physical Review E* 103.6 (Jan. 2021). ISSN: 2470-0053. URL: http://dx.doi.org/10.1103/PhysRevE.103.063302.

[Nak+21] Kouhei Nakaji et al. *Approximate amplitude encoding in shallow parameterized quantum circuits and its application to financial market indicator*. 2021. arXiv: 2103.13211 [quant-ph].